

Arquitectura de Datos

Tema 3: Redis

Bases de datos clave-valor en memoria

Curso académico 2026/2027



T3.1 Introducción a Redis



Resultados de aprendizaje

- a. Comprender qué es Redis y por qué es la base de datos clave-valor más utilizada
- b. Distinguir las características técnicas que hacen único a Redis (monohilo, E/S no bloqueante, estructuras en memoria)
- c. Identificar los casos de uso adecuados e inadecuados para Redis
- d. Conocer las alternativas a Redis y cuándo elegir cada una

Índice de contenidos

T3.1.1 ¿Qué es Redis?	T3.1.5 Operaciones básicas
T3.1.2 Características principales	T3.1.6 Casos de uso fundamentales
T3.1.3 Arquitectura básica	T3.1.7 Cuándo NO usar Redis
T3.1.4 Claves y valores	T3.1.8 Alternativas a Redis

T3.1.1 ¿Qué es Redis?

Redis (Remote Dictionary Server) es un almacén de datos en memoria de código abierto, creado por **Salvatore Sanfilippo** en 2009. Originalmente fue desarrollado para resolver problemas de escalabilidad en una empresa de subastas en línea llamada LLOOGG.

«La simplicidad es la máxima sofisticación.» — Leonardo da Vinci. Redis demuestra que con un modelo de datos simple (clave-valor) se pueden resolver problemas complejos.

La idea surgió de una necesidad real: el sistema necesitaba una forma rápida de almacenar y recuperar datos de sesiones de usuarios concurrentes. Salvatore creó Redis como una alternativa más rápida y flexible a las soluciones existentes como Memcached, que solo soportaba cadenas simples.

Redis no es solo una caché: es un **servidor de estructuras de datos**. A diferencia de Memcached (que solo almacena cadenas), Redis almacena hashes, listas, conjuntos, conjuntos ordenados y estructuras más complejas, todo con operaciones atómicas y de alta performance.

Atomicidad en Redis vs ACID: En Redis, "atómico" significa que un comando se ejecuta de principio a fin sin que ningún otro comando se intercale (garantizado por el hilo único). Pero NO es atómico en sentido ACID: MULTI/EXEC ofrece atomicidad batch (todo o nada), pero NO ofrece aislamiento (otro cliente puede ver estados intermedios entre comandos del batch). Para aislamiento real, usa WATCH + retry.

Redis vs Memcached

Característica	Redis	Memcached
Estructuras de datos	Strings, Hashes, Lists, Sets, Sorted Sets, Bitmaps, HyperLogLog	Solo strings
Persistencia	RDB + AOF	No
Replicación	Sí (primary-replica)	No nativa
Transacciones	Multi/EXEC	No
Publicación/Suscripción	Sí	No
Scripts Lua	Sí	No
Clúster	Sí (nativo)	Client-side sharding

Redis vs bases de datos tradicionales

Característica	Redis	MySQL / MariaDB
Almacenamiento	Memoria RAM	Disco (SSD/HDD)
Velocidad de lectura	~0,1 ms	~1–5 ms
Modelo de datos	Estructuras clave-valor	Tablas con esquema relacional
Transacciones	Multi / EXEC (atómicas)	BEGIN / COMMIT / ROLLBACK
Relaciones	No soporta JOINS	JOINS completos
Consistencia	Eventual (con réplicas)	ACID completa
Caso de uso principal	Caché, sesiones, colas	Datos persistentes, consultas complejas

Consistencia eventual: Tras una escritura, todas las réplicas convergerán al mismo valor, pero no se garantiza cuándo. En Redis con replicación asíncrona, el lag típico es <1ms, pero puede ser mayor bajo carga pesada. Para la mayoría de casos de uso (cachés, sesiones, contadores), consistencia eventual es suficiente. Para pagos o inventario crítico, necesitas consistencia fuerte (quórum).

¿Por qué no se pueden hacer JOINS? Redis almacena datos en claves independientes. No hay relaciones entre claves. Para hacer un JOIN, tendrías que consultar múltiples claves y unir los resultados en la aplicación, lo cual Redis no gestiona.

Conexión con otros temas: En T6 veremos MongoDB (documento) y Cassandra (columnar) como alternativas a Redis para diferentes casos de uso. En T7, Netflix usa Redis/EVCache como caché, Cassandra para el catálogo y Aurora PostgreSQL para transacciones ACID.

Twitter usa Redis como capa de caché principal para timelines, procesando **40–100 millones de comandos por segundo** en sus clústeres Redis. La arquitectura completa de timelines (fan-out, write-behind, replicación) se detalla en T3.2.

Fan-out = distribuir una actualización a múltiples destinatarios. **Fan-out on write:** cuando un usuario publica, escribes en la timeline de TODOS sus seguidores (lectura rápida, escritura lenta). **Fan-out on read:** cuando un usuario consulta su timeline, lees de TODOS los posts de sus seguidores (escritura rápida, lectura lenta).

GitHub utiliza Redis para gestionar el rate limiting de su API. Cada usuario autenticado tiene derecho a 5 000 peticiones por hora. Redis almacena un contador atómico por usuario que se incrementa con cada petición y se resetea automáticamente tras una hora. Esto permite a GitHub escalar su API sin necesidad de base de datos relacional para esta métrica.

Resumen T3.1.1: Redis es un servidor de estructuras de datos en memoria, no solo una caché. Es 100x más rápido que SSD, soporta 5 estructuras principales + módulos, y lo usan Twitter (40-100M cmds/s), GitHub y Netflix.

T3.1.2 Características principales

Almacenamiento en memoria (RAM)

Todos los datos de Redis residen en memoria RAM. Esto es lo que lo hace extremadamente rápido: una operación de lectura o escritura en RAM tarda ~100 nanosegundos, mientras que una operación en disco (SSD) tarda ~100 microsegundos, es decir, **1 000 veces más**.

La consecuencia es que **la capacidad de Redis está limitada por la RAM disponible**. Un servidor con 64 GB de RAM puede almacenar hasta 64 GB de datos Redis (menos el overhead del sistema operativo). Para datasets más grandes, se usa Redis Cluster con múltiples nodos.

Persistencia opcional

A diferencia de Memcached (que pierde todos los datos al reiniciar), Redis ofrece dos mecanismos de persistencia:

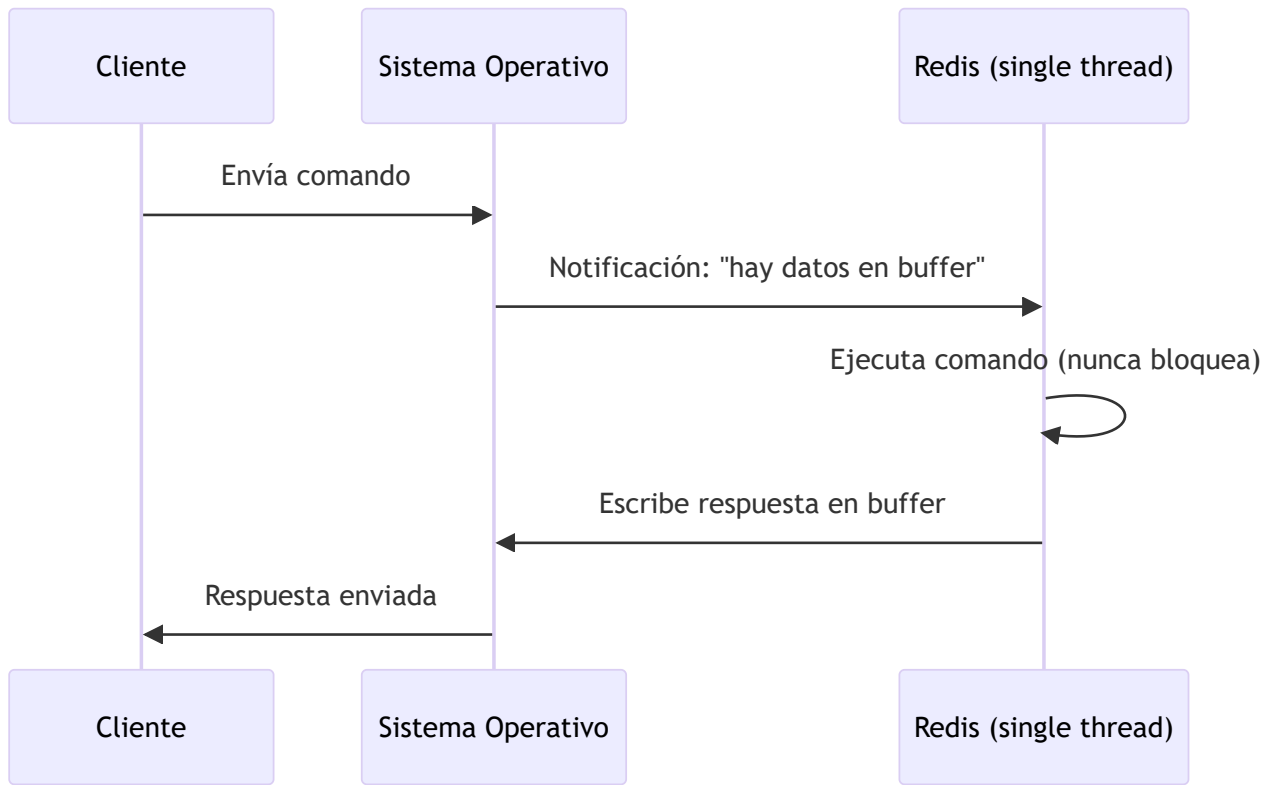
- **RDB (Redis Database):** Snapshots periódicos del dataset completo en un fichero `dump.rdb`. Rápido para backups pero puede perder datos entre snapshots.
- **AOF (Append Only File):** Log de todas las operaciones de escritura. Más seguro (máximo 1 segundo de pérdida con `appendfsync everysec`) pero genera ficheros más grandes.
- **Híbrido (recomendado):** Redis 4.0+ combina RDB + AOF. El AOF se regenera periódicamente a partir del último snapshot RDB.

Monohilo con E/S no bloqueante

Redis utiliza un **único hilo de ejecución** para procesar comandos. Esto parece una limitación, pero en realidad es una ventaja:

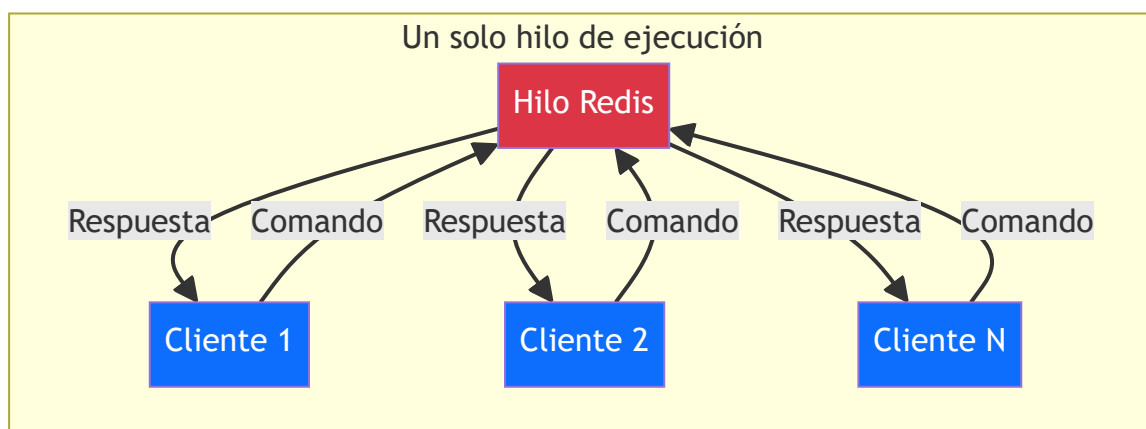
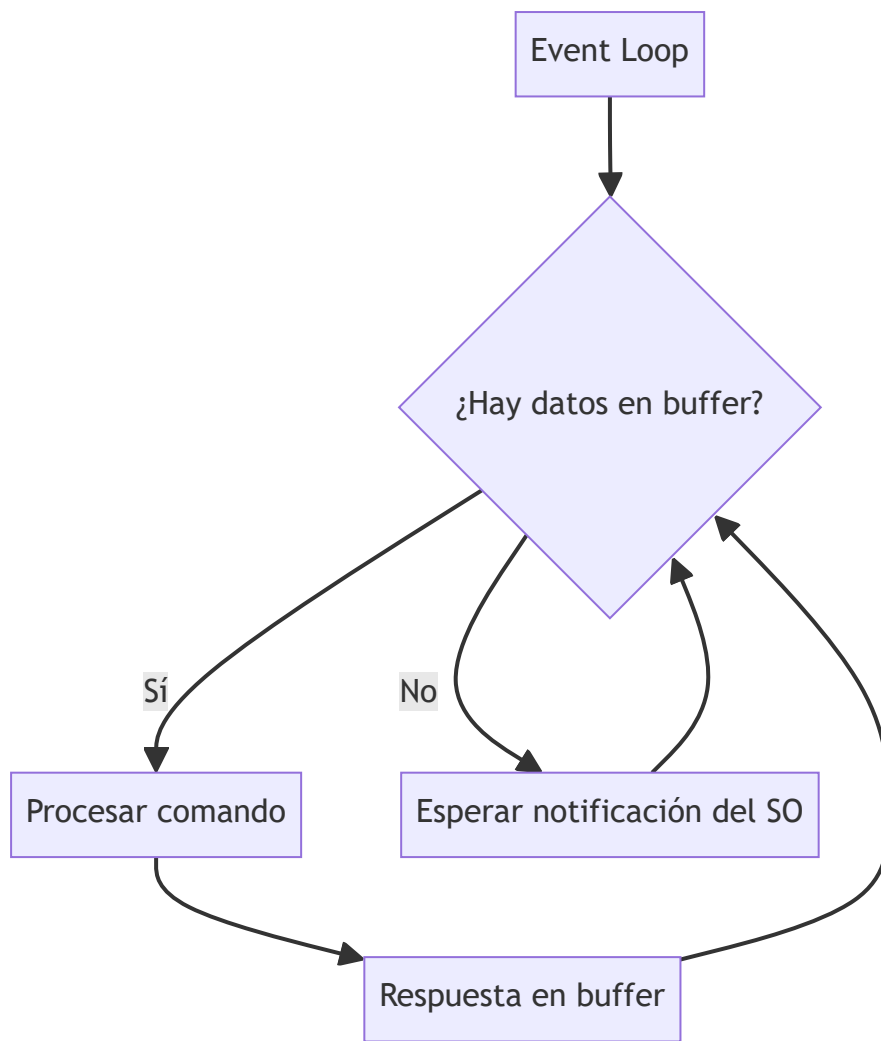
- **No necesita locks:** Como solo un hilo ejecuta comandos, no hay condiciones de carrera ni deadlocks.
- **E/S no bloqueante:** Usa `epoll` (Linux), `kqueue` (macOS) o `IOCP` (Windows) para gestionar miles de conexiones simultáneas sin bloquearse.
- **Operaciones atómicas:** Cada comando se ejecuta de forma atómica e indivisible.

¿Qué es `epoll`/`kqueue`/`IOCP`? Son mecanismos del sistema operativo para E/S no bloqueante. Sin ellos, Redis tendría que preguntar a cada cliente "¿tienes datos?" (polling, ineficiente). Con `epoll` (Linux), `kqueue` (macOS) o `IOCP` (Windows), el SO notifica a Redis cuando hay datos listos en el buffer de red. Redis solo procesa cuando el SO le dice "hay trabajo". Este patrón se llama **Reactor pattern**: un único hilo gestiona miles de conexiones sin bloquearse.



Analogía: Imagina un restaurante con un solo camarero que atiende 100 mesas. Parece imposible, pero si cada orden se resuelve en 100 nanosegundos (como hace Redis), el camarero puede atender a todos sin bloquearse. Un segundo camarero añadiría complejidad de coordinación (locks) sin beneficio real.

¿Por qué funciona con un solo hilo? Si Redis usara E/S bloqueante, cada petición que tardara 1ms en red bloquearía el hilo durante 1ms, procesando solo 1.000 comandos por segundo. Con E/S no bloqueante, el hilo procesa comandos de otros clientes mientras el primer cliente espera su respuesta, consiguiendo más de 100.000 comandos por segundo.



Operaciones $O(1)$ o $O(\log N)$

Todas las operaciones de Redis tienen complejidad temporal garantizada:

Operación	Complejidad	Significado
SET / GET	$O(1)$	Constante: no importa cuántas claves haya
HSET / HGET	$O(1)$	Constante: acceso directo al campo
LPUSH / RPUSH	$O(1)$	Constante: inserción por extremo
SADD	$O(1)$	Constante: inserción en hash table
ZADD	$O(\log N)$	Logarítmica: inserción en skip list
LRANGE	$O(S+N)$	Lineal: recorre la lista
HGETALL	$O(N)$	Lineal: lee todos los campos

Consejo práctico: Usa HGET en lugar de HGETALL cuando solo necesites un campo. HGETALL tiene complejidad $O(N)$ y puede bloquear el servidor si el hash tiene millones de campos.

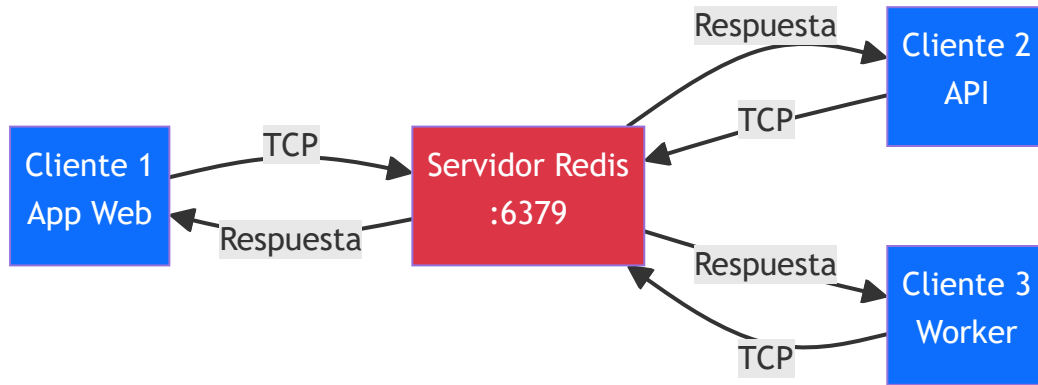
GitHub usa Redis para rate limiting con latencia sub-milisegundo en cada petición API autenticada, permitiendo 5 000 peticiones/hora por usuario. El secreto está en que INCR es $O(1)$ y atómico: miles de usuarios pueden incrementar sus contadores concurrentemente sin bloqueos.

Resumen T3.1.2: Redis almacena en RAM ($\sim 0.1\text{ms}$), tiene persistencia opcional (RDB/AOF), es monohilo con E/S no bloqueante, y todas sus operaciones son $O(1)$ o $O(\log N)$.

T3.1.3 Arquitectura básica

Modelo Cliente-Servidor

Redis sigue un modelo **cliente-servidor** donde múltiples clientes se conectan a un único servidor Redis a través de TCP. El servidor procesa los comandos de forma **secuencial** (monohilo), pero de forma **no bloqueante**, lo que le permite gestionar miles de conexiones simultáneas.



¿Por qué funciona con un solo hilo?

La clave está en que Redis **no bloquea en E/S de red**. Utiliza mecanismos del sistema operativo como `epoll` (Linux) o `kqueue` (macOS) que permiten gestionar miles de conexiones de forma eficiente sin crear un hilo por conexión.

El flujo es:

1. Un cliente envía un comando a Redis.
2. Redis lee el comando del buffer de red (no bloquea).
3. Redis ejecuta el comando (todo en memoria, ~100ns).
4. Redis escribe la respuesta en el buffer de red del cliente.
5. El cliente recibe la respuesta.

Ninguno de estos pasos bloquea el hilo principal. Mientras un cliente espera su respuesta, Redis procesa comandos de otros clientes.

Protocolo RESP (REdis Serialization Protocol)

Redis utiliza el protocolo **RESP** para comunicarse con los clientes. Es un protocolo simple, binario seguro y fácil de implementar. Existen tres versiones:

Contenido avanzado / opcional: El protocolo RESP es interesante para entender cómo Redis se comunica en la red, pero no es necesario para usar Redis. Si el ritmo de clase va rápido, puedes profundizar aquí. Si no, continúa con la siguiente sección.

```

# RESP2 – Comando simple (string)
*2
$4
PING
# Respuesta:
+PONG

# RESP2 – SET con TTL
*5
$3
SET
$7
mi_clave
$5
valor
$2
EX
$2
60
# Respuesta:
+OK

# RESP2 – Respuesta numérica
*2
$4
LLEN
$5
mi_lista
# Respuesta:
:42

# RESP2 – Respuesta de error
*2
$4
GET
$7
mi_clave
# Respuesta:
-ERR wrong number of arguments for 'get' command

```

Instagram usa Redis con múltiples instancias para contadores de likes y Stories, explotando la simplicidad del modelo monohilo para conseguir latencia consistente. Su arquitectura usa Redis como capa de caché delante de Cassandra, con un patrón Write-Behind que permite escrituras instantáneas en Redis y persistencia asíncrona en Cassandra.

Resumen T3.1.3: Redis usa un modelo cliente-servidor monohilo. No bloquea porque usa epoll/kqueue para E/S no bloqueante. Se comunica por el protocolo RESP (simple y binario seguro).

T3.1.4 Claves y valores

Convenciones de nombrado

Redis almacena datos en **claves** (keys) que asocian **valores** (values). Las claves son cadenas de texto que siguen convenciones de nombrado para organizar los datos. El patrón más habitual usa tres partes separadas por dos puntos (:):

TTL (Time To Live): Cada clave puede tener un tiempo de vida en segundos. Tras expirar, Redis la elimina automáticamente. Es la base de las cachés: guardas un dato con TTL=3600 (1 hora), y cuando caduca, lo vuelves a cargar de la fuente original.

Las 3 partes de una clave Redis:

Parte Redis	Equivalente en SQL	Ejemplo	¿Qué es?
Objeto	Tabla	user	El tipo de entidad que almacenas
Identificador	ID de fila (PRIMARY KEY)	1000	El registro concreto dentro de esa entidad
Campo	Columna	name	La propiedad específica que guardas

```
# Patrón: objeto:identificador:campo
#
SET user:1000:name "Juan García"
#
#   objeto identificador campo

SET user:1000:email "juan@email.com"
SET session:abc123:userId "1000"
SET product:456:price "29.99"

# Patrón: objeto:identificador (para hashes)
# Un solo Hash agrupa todos los campos de una entidad
HSET user:1000 name "Juan García"
HSET user:1000 email "juan@email.com"

# Patrón: prefijo:tipo:dato
SET cache:product:456 '{"name":"iPhone","price":999}'

# Patrón con namespaces (recomendado en producción)
#.Namespace añade contexto de aplicación
SET app:shop:product:456:info "..."
SET app:shop:session:abc123:userId "1000"
```

SQL vs Redis — la misma lógica, diferente formato:

SQL: SELECT name FROM user WHERE id = 1000

Redis: GET user:1000:name

En SQL la fila 1000 de la tabla user tiene una columna name. En Redis, la clave user:1000:name es exactamente lo mismo: el objeto (user), su ID (1000), y el campo (name). La diferencia es que en Redis no hay esquema — las claves son cadenas de texto, no columnas definidas en una tabla.

SQL (MariaDB)	Redis	Valor
Tabla products	Objeto product	—
Fila con id=456	Identificador 456	—
Columna name	Campo name	"iPhone 15"
Columna price	Campo price	"999"
product:456:name = "iPhone 15" → es como hacer SELECT name FROM products WHERE id=456		

Estructura interna de las claves

Internamente, Redis almacena las claves en una **hash table global** (diccionario). Cuando el número de claves es pequeño, usa un **listpack** (lista compacta). Cuando crece, rehashing a una hash table completa de forma asíncrona (rehashing incremental).

Límites de Redis

Elemento	Límite máximo	Nota
Tamaño de clave	512 MB	En la práctica, usar claves < 1 KB
Tamaño de valor	512 MB	Strings: 512 MB. Listas: $2^{32} - 1$ elementos
Número de claves	Sin límite práctico	Depende de la RAM disponible
Niveles de anidamiento	Sin límite	Se recomienda < 6 niveles para rendimiento
Número de campos en hash	$2^{32} - 1$ (≈4 mil millones)	En la práctica, < 100 000 campos

Convenciones de nombrado en producción

- **Usa dos puntos (:)** como separador: user:1000:profile
- **Evita espacios y caracteres especiales:** No uses espacios, comillas, o saltos de línea en claves.
- **Usa namespaces:** Prefijo de aplicación: app:shop:user:1000
- **Sé consistente:** Si usas user:1000:name, no cambies a users.1000.name
- **Evita claves demasiado largas:** Cada byte de clave consume memoria. Claves de 100 bytes son razonables.

- **No uses KEYS en producción:** Es $O(N)$ y bloquea el servidor. Usa SCAN en su lugar.

Discord usa claves con patrón chat: `{channel_id}` para particionar datos de chat entre múltiples instancias Redis, consiguiendo latencia < 1 ms para mensajes en tiempo real. Cada canal de chat tiene su propia clave, lo que permite distribuir la carga de forma natural entre nodos del clúster.

Resumen T3.1.4: Usa dos puntos como separador (user:1000:name). El límite es 512 MB por clave y valor. Usa namespaces (app:shop:...). En producción, nunca uses KEYS, usa SCAN.

T3.1.5 Operaciones básicas

CRUD en Redis

Operaciones CRUD básicas

```
# === CREATE - Crear / actualizar claves ===
SET user:1000 "Juan García"
SETEX session:abc123 1800 "userId:1000" # Con TTL de 30 minutos
SETNX lock:resource1 "owner-123" # Solo si no existe (lock)

# === READ - Leer claves ===
GET user:1000 # Una clave
MGET user:1000 user:1001 user:1002 # Lectura múltiple (más eficiente)

# === UPDATE - Actualizar (sobreescribe) ===
SET user:1000 "Juan García Updated" # Sobreescribe el valor anterior

# === DELETE - Eliminar claves ===
DEL user:1000 # Eliminación síncrona
UNLINK user:1001 # Eliminación asíncrona (no bloqueante, Redis 4.0+)
```

Equivalencia Redis vs SQL

Para quien viene de bases de datos relacionales, esta tabla muestra la correspondencia directa entre comandos Redis y SQL:

Operación	SQL (MariaDB)	Redis	Ejemplo Redis
Crear registro	INSERT INTO users (id, name, email) VALUES (1000, 'Juan', 'juan@mail.com')	SET	SET user:1000:name "Juan" SET user:1000:email "juan@mail.com"
Leer un registro	SELECT name FROM users WHERE id = 1000	GET	GET user:1000:name
Leer varios registros	SELECT name, email FROM users WHERE id IN (1000, 1001, 1002)	MGET	MGET user:1000:name user:1001:name user:1002:name
Actualizar campo	UPDATE users SET name = 'Juan García' WHERE id = 1000	SET	SET user:1000:name "Juan García"
Eliminar registro	DELETE FROM users WHERE id = 1000	DEL	DEL user:1000:name
Verificar existencia	SELECT COUNT(*) FROM users WHERE id = 1000	EXISTS	EXISTS user:1000:name
Incrementar contador	UPDATE users SET views = views + 1 WHERE id = 1000	INCR	INCR user:1000:views
Establecer expiración	– (no tiene equivalente directo)	EXPIRE	EXPIRE session:abc123 1800
Listar todas las claves	SHOW TABLES	SCAN	SCAN 0 MATCH user:* COUNT 100

Diferencia clave: En SQL, INSERT crea y UPDATE modifica. En Redis, SET hace ambas cosas — si la clave ya existe, la sobrescribe. No hay distinción entre crear y actualizar.

Gestión de claves

Gestión y metadatos de claves

```
# Verificar existencia (devuelve número de claves que existen)
EXISTS user:1000          # 1 si existe, 0 si no
EXISTS user:1000 user:9999 # 1 (solo una existe)

# Obtener tipo de dato
TYPE user:1000            # "string", "hash", "list", "set", "zset"

# Obtener TTL (Time To Live)
TTL session:abc123        # -1 = sin expiración, -2 = no existe, >0 = segundos restantes
PTTL session:abc123       # TTL en milisegundos

# Buscar claves por patrón (¡CUIDADO en producción!)
KEYS user:*               # O(N) – bloquea el servidor si hay millones de claves
SCAN 0 MATCH user:* COUNT 100 # Seguro e iterativo – recomendado

# Renombrar una clave
RENAME user:1000 user:1000:v2

# Obtener tamaño de una clave (en bytes)
MEMORY USAGE user:1000
```

⚠ Advertencia: El comando KEYS es O(N) y bloquea el servidor Redis durante toda la operación. En producción con millones de claves, puede causar tiempos de espera en todos los clientes conectados. Usa SCAN que es iterativo y no bloquea.

Producción: En producción, usa SCAN en lugar de KEYS. SCAN es iterativo y no bloquea el servidor, mientras que KEYS puede congelar Redis durante varios segundos con millones de claves.

Expiración de datos

Gestión de TTL

```
# Establecer expiración en segundos
EXPIRE clave 600           # 10 minutos
PEXPIRE clave 1800000      # 30 minutos en milisegundos

# Establecer expiración con timestamp Unix
EXPIREAT clave 1726400000  # Expira en timestamp específico

# Establecer valor CON expiración de una vez
SETEX clave 600 "valor"    # SET + EXPIRE en una operación atómica
SET clave "valor" EX 600   # Forma moderna (Redis 2.6.12+)
SET clave "valor" PX 1800000 # En milisegundos

# Eliminar expiración
PERSIST clave              # Quitar TTL, la clave ya no expira
```

Operaciones atómicas

Incremento, decremento y operaciones atómicas

```
# Incremento / decremento atómico (funciona con strings numéricos)
SET contador 0
INCR contador           # 1
INCR contador           # 2
INCRBY contador 5       # 7
DECR contador           # 6
DECRBY contador 3       # 3

# Incremento de punto flotante
SET precio 19.99
INCRBYFLOAT precio 0.5   # 20.49
INCRBYFLOAT precio -1.0  # 19.49

# Incrementar campo de un hash
HSET user:1000 login_count 0
HINCRBY user:1000 login_count 1
HINCRBY user:1000 login_count 1
```

Transacciones

Redis permite agrupar múltiples comandos en una **transacción atómica** con MULTI/EXEC. Todos los comandos se ejecutan de forma serial sin interrupciones:

```
# Iniciar transacción
MULTI

# Encolar comandos (NO se ejecutan aún)
SET user:1000:nombre "Juan"
SET user:1000:email "juan@email.com"
INCR estadisticas:usuarios

# Ejecutar todos los comandos de una vez
EXEC

# Respuesta:
# 1) OK
# 2) OK
# 3) (integer) 1

# Descartar transacción (si cambias de idea)
MULTI
SET temp "data"
DISCARD
```

Diferencia con INCR: MULTI/EXEC garantiza que los comandos se ejecutan de forma serial, pero **no** proporciona aislamiento (como en SQL). Si necesitas aislamiento real, usa WATCH para optimistic locking.

Pipeline (agrupar comandos)

Si necesitas enviar múltiples comandos sin necesidad de transacción, usa **pipeline**. Agrupa comandos en un solo roundtrip a la red, reduciendo latencia de forma drástica:

```
# Sin pipeline: 3 roundtrips a la red
SET user:1:name "Juan"
SET user:1:email "juan@email.com"
SET user:1:age 28

# Con pipeline: 1 solo roundtrip
PIPELINE
SET user:1:name "Juan"
SET user:1:email "juan@email.com"
SET user:1:age 28
EXEC

# En producción (pseudocódigo de la aplicación):
# pipe = redis.pipeline()
# for user in users:
#     pipe.set(f"user:{user.id}:name", user.name)
# pipe.execute() # Un solo roundtrip para N comandos
```

Pipeline vs MULTI/EXEC: Pipeline agrupa comandos para reducir latencia (red). MULTI/EXEC garantiza ejecución atómica sin interrupciones. Puedes usar ambos juntos si necesitas atomicidad Y rendimiento.

Impacto del Pipeline: Sin pipeline: 1000 comandos = 1000 roundtrips (ida y vuelta por la red) $\approx$ 500ms con latencia de 0.5ms por roundtrip. Con pipeline: 1000 comandos = 1 roundtrip $\approx$ 0.5ms. Mejora de 1000x. El cliente acumula comandos en un buffer y los envía todos juntos; Redis los ejecuta y responde todos juntos.

WATCH (Optimistic Locking)

WATCH permite **optimistic locking**: monitorizar una clave y abortar la transacción si alguien la modifica entre WATCH y EXEC:

```

# Cliente 1: Lee el saldo actual
WATCH cuenta:saldo
# Respuesta: (integer) 1000

# Otro cliente modifica la clave mientras tanto:
# SET cuenta:saldo 800

# Cliente 1 intenta actualizar
MULTI
DECRBY cuenta:saldo 200
EXEC
# Respuesta: (nil) ← La transacción fue abortada porque la clave cambió

# Cliente 1 debe reintentar:
WATCH cuenta:saldo
# Leer nuevo valor: 800
MULTI
DECRBY cuenta:saldo 200
EXEC
# Respuesta: (integer) 1 ← Ahora sí se ejecuta

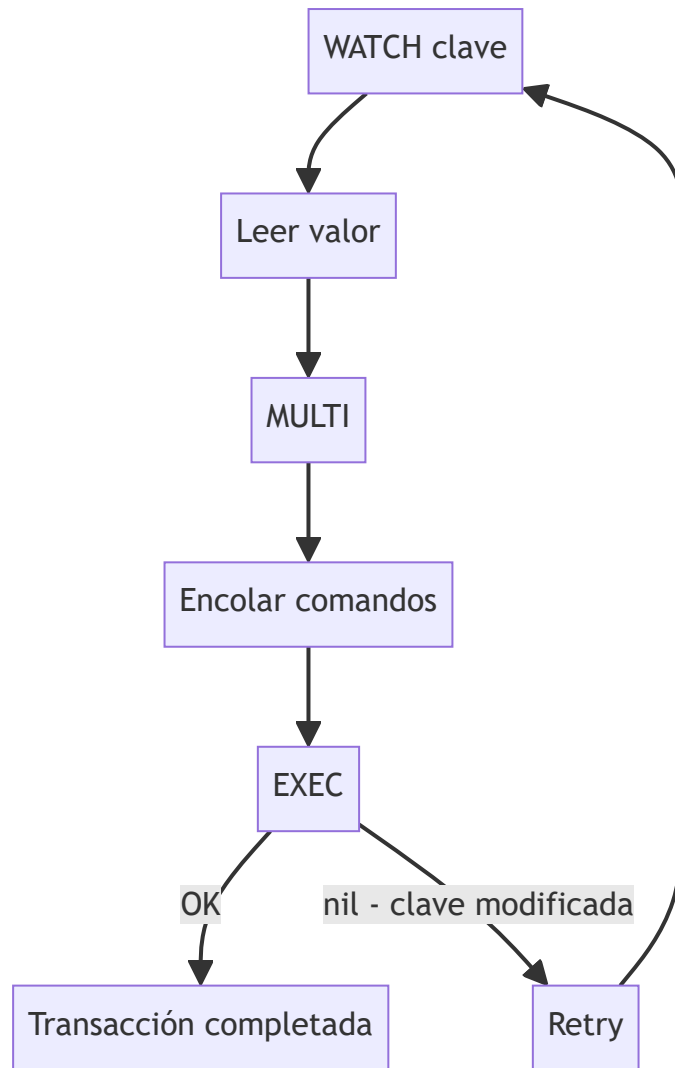
```

¿Cuándo usar WATCH? Cuando necesitas leer-modificar-escribir sin perder datos por concurrencia. Es el equivalente a un `SELECT ... FOR UPDATE` en SQL, pero sin bloqueos pesados.

Pessimista vs Optimista:

Pessimista (SELECT FOR UPDATE): bloquea la fila ANTES de leer. Otros esperan. Útil cuando los conflictos son frecuentes.

Optimista (WATCH): no bloquea nada. Al escribir, verifica que el valor no cambió. Si cambió → retry. Útil cuando los conflictos son infrecuentes.



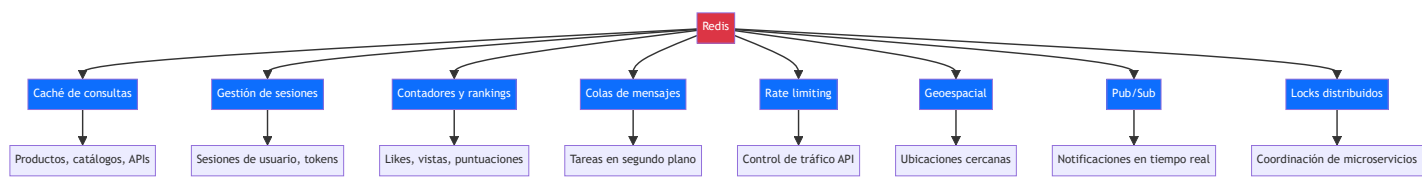
Caso real — Transferencia bancaria: WATCH balance_cuenta_A, leer saldo, verificar suficiente, MULTI, DEBITO A, CREDITO B, EXEC. Si otro proceso modificó el saldo entre WATCH y EXEC → retry.

¿Por qué INCR es atómico? Redis ejecuta todos los comandos en un único hilo. Cuando ejecuta INCR, nadie más puede ejecutar comandos hasta que termine. Esto garantiza que dos clientes concurrentes nunca obtienen el mismo resultado. Es la misma propiedad que hacen los bancos para contadores de transacciones.

Netflix usa EXPIRE en sesiones de reproducción para limpiar automáticamente datos transitorios después de cada sesión. Cuando un usuario cierra la app, su sesión en Redis expira automáticamente tras 30 minutos, liberando memoria sin necesidad de un proceso de limpieza externo.

Resumen T3.1.5: CRUD con SET/GET/DEL. TTL con EXPIRE. INCR es atómico y $O(1)$, ideal para contadores. UNLINK es la versión asíncrona de DEL.

T3.1.6 Casos de uso fundamentales



Caché de consultas

El caso de uso más común. Redis almacena resultados de consultas frecuentes (productos, perfiles, configuraciones) para evitar consultar la base de datos principal.

Ejemplo: Caché de producto

```
# Consulta frecuente: información de producto
# Sin caché: SELECT * FROM products WHERE id=456 (5ms en MariaDB)
# Con caché: GET product:456:info (0.1ms en Redis)

SET product:456:info '{"name":"iPhone 15","price":999,"stock":50}' EX 300
GET product:456:info

# Cuando el producto se actualiza, invalidar la caché
# UPDATE products SET price=899 WHERE id=456;
DEL product:456:info
```

Gestión de sesiones

Redis es ideal para sesiones de usuario porque permite acceso rápido y expiración automática.

Ejemplo: Sesión de usuario

```
# Login: crear sesión
SET session:xyz789:userId "1000" EX 1800
SET session:xyz789:role "admin" EX 1800
SET session:xyz789:lastAccess "2026-09-15T10:30:00" EX 1800

# Verificar sesión en cada petición
GET session:xyz789:userId

# Renovar sesión (sliding expiration)
EXPIRE session:xyz789:userId 1800

# Logout: eliminar sesión
DEL session:xyz789:userId
DEL session:xyz789:role
DEL session:xyz789:lastAccess
```

Contadores y rankings

Las operaciones INCR y los Sorted Sets son perfectos para métricas en tiempo real.

Ejemplo: Contadores y ranking

```
# Contador de vistas de producto
INCR views:product:456
INCR views:product:456
INCR views:product:456    # 3 vistas

# Contador de likes
INCR likes:post:789

# Ranking de productos más vendidos
ZADD top:sales 150 "product:456"
ZADD top:sales 230 "product:789"
ZADD top:sales 89 "product:123"
ZREVRANGE top:sales 0 2 WITHSCORES    # Top 3
```

Colas de mensajes

Redis puede usarse como cola de mensajes simple con LPUSH + BRPOP.

Ejemplo: Cola de tareas

```
# Productor: añadir tareas
LPUSH cola:tareas '{"task":"send_email","to":"user@email.com"}'
LPUSH cola:tareas '{"task":"process_image","id":456}'

# Consumidor: extraer tareas (bloqueante)
BRPOP cola:tareas 0 # Espera hasta que haya una tarea

# Si necesitas colas con prioridad:
LPUSH cola:alta '{"priority":"high","task":"..."}'
LPUSH cola:baja '{"priority":"low","task":"..."}'
```

Rate limiting

Controlar la tasa de peticiones de clientes para evitar abusos.

Ejemplo: Rate limiting simple

```
# Rate limiting con ventana fija (100 peticiones/hora)
SET ratelimit:user:1000 0 EX 3600 # Contador con TTL de 1 hora

# En cada petición:
INCR ratelimit:user:1000
# Si el resultado > 100 → rechazar petición (HTTP 429)
```

Twitch usa Redis para caché de chat en tiempo real, gestionando millones de mensajes por minuto. Durante *Twitch Plays Pokémon*, el sistema recibió 1,2 millones de mensajes en 24 horas. El problema era que una sola instancia Redis no podía con la carga. La solución fue shard por canal de chat: `chat:{channel_id}` se distribuye entre múltiples instancias Redis, permitiendo escalar horizontalmente.

Resumen T3.1.6: Los casos principales son: caché de consultas, sesiones, contadores, colas (LPUSH+BRPOP) y rate limiting. Cada caso usa una estructura diferente de Redis.

T3.1.7 Cuándo NO usar Redis

Redis es una herramienta excelente, pero no es la solución para todo. Un buen ingeniero debe saber **cuándo no usarla**:

Limitación	Explicación	Alternativa
Coste de RAM	~1 €/GB en RAM vs ~0,1 €/GB en SSD. Almacenar 100 GB en Redis cuesta ~100 €/mes vs ~10 €/mes en disco.	Usar caché selectiva: solo datos calientes en Redis.
Sin consultas complejas	No soporta JOINS, GROUP BY, subqueries, ordenaciones complejas.	Usar Redis para caché y MariaDB/PostgreSQL para consultas.
Límite de 512 MB por valor	Un string, hash o lista no puede superar 512 MB.	Particionar datos en múltiples claves.
Persistencia no ACID	RDB puede perder datos entre snapshots. AOF es mejor pero no garantiza consistencia como una BD relacional.	Usar Redis como caché, no como fuente primaria de datos.
Monohilo en escritura	Las escrituras secuenciales están limitadas por un solo hilo.	Para escritura masiva, considerar Apache Kafka o bases de datos por columnas.
No reemplaza una BD relacional	Redis no tiene transacciones ACID completas, no soporta relaciones, no tiene LDD.	Usar Redis como complemento, no como sustituto.

Regla de oro: Redis es un **complemento** de tu base de datos principal, no un **sustituto**. Úsalo para caché, sesiones, contadores y colas. Deja la persistencia y las consultas complejas a la BD relacional.

Redis NO es una base de datos relacional. No tiene JOINS, transacciones ACID completas, ni integridad referencial. Es una estructura de datos en memoria optimizada para velocidad. Úsalo como caché o almacén de datos calientes, no como BD principal.

Caso real: Twitch — Twitch Plays Pokémon

En 2016, Twitch lanzó "Twitch Plays Pokémon", un experimento donde millones de espectadores enviaban comandos de chat simultáneamente. El sistema Redis que gestionaba el chat colapsó.

Problema:

- 1,2 millones de mensajes de chat en 24 horas.
- CPU de Redis al 100% (single-threaded, un solo núcleo saturado).
- 3.021 conexiones clientes activas, 2.311 en espera.
- Una sola instancia Redis para todo el chat de Twitch.

Solución:

- **Shard por clave:** `chat:{channel_id}` → cada canal va a una instancia Redis diferente.
- **HAProxy:** balanceo de carga entre particiones Redis.
- **Reducción de conexiones:** cerrar conexiones en espera innecesarias.

Lección: Redis es single-threaded por diseño. Cuando un solo nodo se satura, la solución no es optimizar el código, sino **particionar los datos** en múltiples instancias. Esto es exactamente lo que resuelve Redis Cluster.

T3.1.8 Alternativas a Redis

Redis no es la única opción para caché en memoria. Desde 2024, con el cambio de licencia de Redis (de BSD a RSAL/SSPL), han surgido alternativas importantes:

Licencia: En marzo de 2024, Redis Ltd. cambió su licencia de BSD a RSALv2/SSPL, lo que provocó la creación de Valkey (fork de Linux Foundation). Esto no afecta a Redis como herramienta, pero sí a quién puede ofrecerlo como servicio gestionado.

Solución	Arquitectura	Licencia	Rendimiento	Cuándo usarla
Redis 8	Single-thread + bg helpers	RSAL/SSPL/AGPL	~250K ops/s (8 cores)	Estándar de facto, máximo ecosistema
Valkey 8	Single-thread + I/O async	BSD-3 (Linux Foundation)	~700K ops/s (8 cores)	Reemplazo directo, open source, soportado por AWS/GCP
Dragonfly	Shared-nothing por core	BSL 1.1 → Apache 2.0	~3.8M ops/s (8 cores)	Máximo rendimiento, 80% menos memoria
KeyDB	Multi-threaded fork	BSD-3	~1M ops/s (8 cores)	Replicación multi-master, FLASH tiering
Memcached	Multi-threaded, simple KV	BSD revisado	Muy alto (simple)	Solo caché simple sin estructuras de datos
Garnet (Microsoft)	C# async, RESP-compatible	MIT	~2.5M ops/s	Ecosistema .NET, compatibilidad Redis

¿Qué elegir? Para la mayoría de proyectos: **Valkey** (open source, compatible al 100% con Redis, soportado por AWS/GCP). Para máximo rendimiento de un solo nodo: **Dragonfly**. Para caché pura sin estructuras: **Memcached**. Redis sigue siendo el estándar de facto con mayor ecosistema y documentación.

☁ **AWS y Valkey:** Desde finales de 2024, AWS ElastiCache usa Valkey por defecto para nuevos despliegues. Google Cloud MemoryStore también migró a Valkey en 2025. Si despliegas en la nube, es probable que ya estés usando Valkey sin saberlo.

Concepto clave: La compatibilidad entre Redis, Valkey, Dragonfly y KeyDB es casi total — hablan el mismo protocolo (RESP). Cambiar de uno a otro es un cambio de configuración, no de código. Lo que cambia es la licencia, el rendimiento de un solo nodo y el soporte de módulos.

Cuando lo pongas en práctica

- Empieza siempre con `redis-cli ping` para verificar la conexión antes de cualquier operación.
- Usa `CONFIG GET *` para ver toda la configuración actual y familiarizarte con las opciones disponibles.
- En producción, nunca uses `KEYS *` — usa `SCAN` con cursor para iterar claves sin bloquear el servidor.
- Configura `maxmemory` y `maxmemory-policy` desde el primer día para evitar errores inesperados.
- Monitoriza con `INFO stats` periódicamente para detectar problemas antes de que afecten a los usuarios.

Preguntas para reflexionar

Pregunta 1

Instagram usa `INCR` para likes y cada like se escribe en Cassandra de forma asíncrona (Write-Behind). Si Redis cae 5 segundos, ¿se pierden likes? ¿Es un problema real? ¿Qué propiedad de `INCR` garantiza que dos usuarios concurrentes nunca obtienen el mismo resultado?

Pregunta 2

Redis se ejecuta en un solo hilo pero es extremadamente rápido. ¿Por qué un diseño monohilo puede ser una ventaja en un sistema de caché? ¿En qué escenarios sería un cuello de botella?

Pregunta 3

Si Redis pierde datos cuando falla (sin persistencia), ¿por qué empresas como Twitter y GitHub lo usan en producción? ¿Qué datos justifican el riesgo y cuáles no?



Arquitectura de Datos

4.º Curso, Grado en Ingeniería Informática

Curso académico 2026/2027

Human Language and Accessibility Technologies Group (HULAT)

Departamento de Informática

Universidad Carlos III de Madrid